Multithreading +
Alte Concepte

Multithreading

Introducere în Multithreading în C++

Multithreading este capacitatea unui program de a executa simultan mai multe părți ale codului său. Aceste părți sunt numite fire de execuție (threads).

Firele de execuție partajează același spațiu de memorie al unui proces, permițându-le să comunice și să partajeze date mai ușor decât procesele separate.

Avantaje Multithreading

Performanță Îmbunătățită: Prin utilizarea mai multor nuclee ale CPU-ului, multithreading-ul poate reduce semnificativ timpul de execuție al sarcinilor intensive din punct de vedere computațional.

Reactivitate Sporită: În aplicațiile GUI, multithreading-ul previne blocarea interfeței grafice în timpul operațiilor de lungă durată.

Utilizarea Mai Bună a Resurselor: Firele de execuție sunt mai ușoare comparativ cu procesele, ceea ce duce la o utilizare mai eficientă a resurselor sistemului.

Concepte Fundamentale 1

Threads vs. Processes:

Procesele sunt medii de execuție independente, cu propriul spațiu de memorie.

Firele de execuție există în interiorul unui proces și partajează spațiul său de memorie.

Concepte Fundamentale 2

Concurență vs. Paralelism

Concurență: Mai multe sarcini avansează, dar nu neapărat în același moment.

Paralelism: Mai multe sarcini se execută simultan pe procesoare diferite.

Ciclu de viață al unui Thread

Nou (New): Firul de execuție este creat.

Rulabil (Runnable): Firul de execuție este gata de rulare.

În Rulare (Running): Firul de execuție se execută în prezent.

Blocat/În Așteptare (Blocked/Waiting): Firul de execuție așteaptă o resursă sau un eveniment.

Terminat (Terminated): Firul de execuție și-a finalizat execuția.

Crearea Firelor de Execuție în C++

Biblioteca Standard C++: C++11 a introdus header-ul <thread> pentru gestionarea firelor de execuție.

```cpp
#include <iostream>
#include <thread>
#include <chrono> // pentru std::this_thread::sleep_for

void task1() {
    std::cout << "Execut task1 in thread: " << std::this_thread::get_id() << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(2)); // Simulăm o operație mai lungă
    std::cout << "task1 gata!" << std::endl;
}

void task2(int x) {
    std::cout << "Execut task2 cu x = " << x << " in thread: " << std::this_thread::get_id() << std::endl;
}

int main() {
    std::thread t1(task1);
    std::thread t2(task2, 10);

    t1.join();
    t2.join();

    std::cout << "Firul principal continuă..." << std::endl;
    return 0;
}
```

Gestionare fire de execuție

Așteptarea Terminării Firelor de Execuție (**Joining**): Metoda join() asigură că firul principal așteaptă finalizarea firelor create.

Detașarea Firelor de Execuție (**Detaching**): Metoda detach() permite unui fir de execuție să ruleze independent în fundal. Folosiți cu precauție!

ID-ul Firului de Execuție: std::this_thread::get_id() returnează ID-ul firului de execuție curent.

Partajarea Datelor și Sincronizarea

Condiții de Cursă (**Race Conditions**): Atunci când mai multe fire de execuție accesează și modifică date partajate concurent, pot apărea rezultate imprevizibile și incorecte.

Excludere Mutuală (**Mutex**): Un mutex (mutual exclusion) este o primitivă de sincronizare care permite unui singur fir de execuție să acceseze o resursă partajată la un moment dat.

Example mutex

```cpp
#include <iostream>
#include <thread>
#include <mutex>
#include <vector>
#include <numeric>

std::mutex mtx;
std::vector<int> numbers;

void add_numbers(int start, int end) {
    for (int i = start; i <= end; ++i) {
        std::lock_guard<std::mutex> lock(mtx);
        numbers.push_back(i);
    }
}

int main() {
    std::thread t1(add_numbers, 1, 50000);
    std::thread t2(add_numbers, 50001, 100000);

    t1.join();
    t2.join();

    long long sum = std::accumulate(numbers.begin(), numbers.end(), 0LL);
    std::cout << "Suma numerelor: " << sum << std::endl;
    std::cout << "Numar elemente: " << numbers.size() << std::endl;

    return 0;
}
```

Probleme care pot să apară

Blocaje Reciproce (**Deadlocks**): Atunci când două sau mai multe fire de execuție sunt blocate indefinit, așteptând reciproc una pe cealaltă.

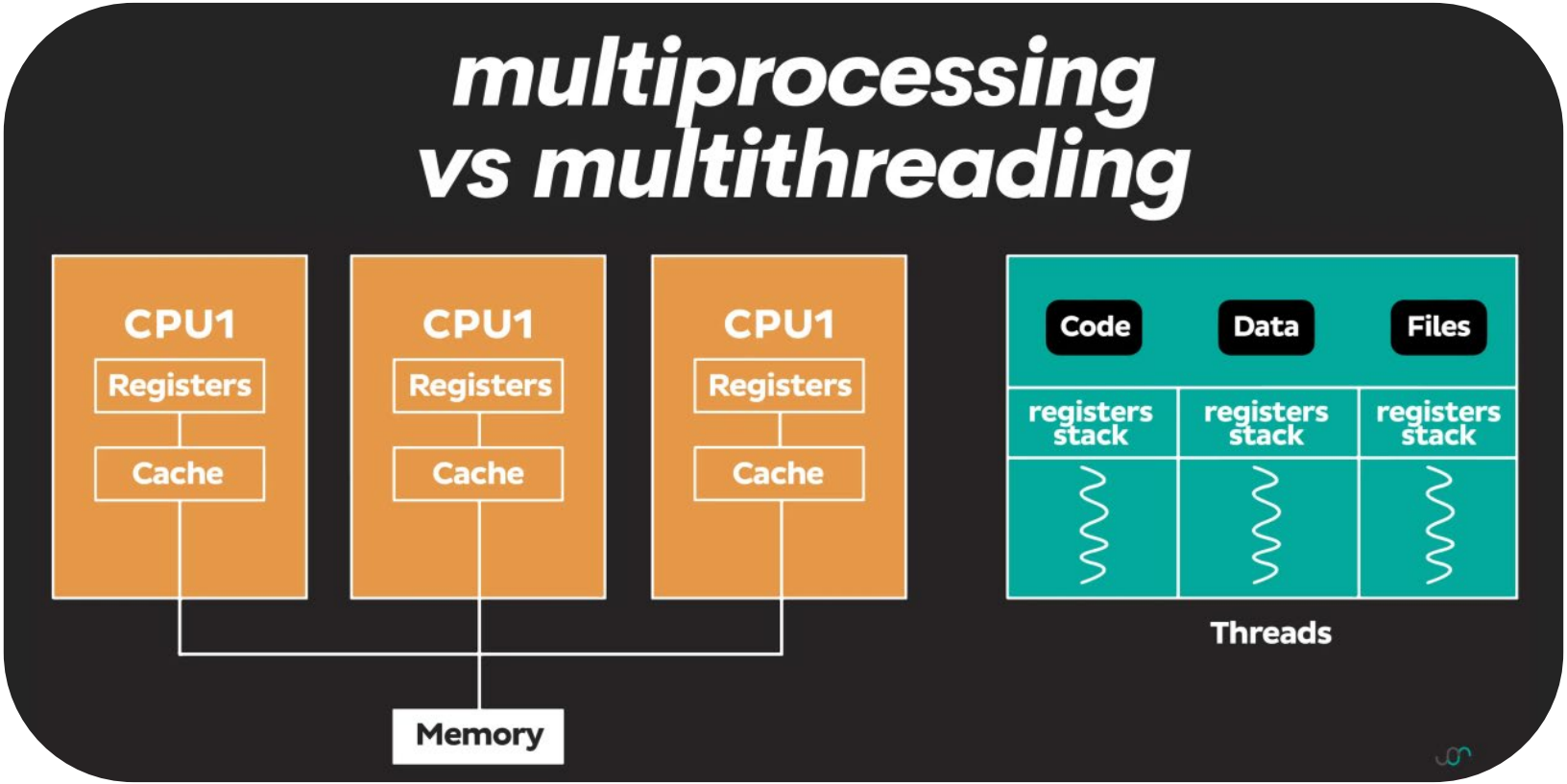
Condiții de Cursă pe Date (**Data Races**): Atunci când mai multe fire de execuție accesează date partajate concurent, și cel puțin un acces este o scriere, fără o sincronizare adecvată.

Summary

Multithreading-ul este o tehnică puternică pentru îmbunătățirea performanței și a reactivității aplicațiilor.

C++ oferă instrumente robuste pentru gestionarea și sincronizarea firelor de execuție.

Trebuie acordată o atenție deosebită partajării și sincronizării datelor pentru a evita probleme potențiale precum condițiile de cursă și blocajele reciproce.


multiprocessing
vs multithreading
CPU1
Registers
Cache
CPU1
Registers
Cache
CPU1
Registers
Cache
Memory
Code
Data
Files
registers stack
registers stack
registers stack
Threads

Avantaje față de C

C vs. C++

C++ a apărut la 12 ani după C, în 1985, drept un răspuns la multele neajunsuri și dificultăți pe care le impune în utilizarea sa.

În ciuda vechimii sale, limbajul încă se bucură de suport (ajungând în 2023 la standardul C++ 23) și rămâne un limbaj relevant pe piață, în sectoare precum grafică, fintech sau embedded systems.

Comparativ cu C, vine în ajutorul programatorului cu multe funcționalități ce îi pot face viața semnificativ mai ușoară.

STL

STL

Unul dintre feature-urile emblematice ale C++, Standard Template Library este un set de template-uri ce implementează algoritmi și structuri de date, scutindu-ne de efortul făcut pentru a le implementa manual.

Funcționalitățile din STL pot fi apelate prin **std::**, sau utilizând **namespace std**.

Exemple:

Queue

```
queue<int> q1;

q1.push(10);

while (!q1.empty()) {
    cout << q1.front() << endl;
    q1.pop();
}
```

Vector, Sort, Auto

```
std::vector<vector<int>> v =
    {{3, 1, 2}, {6, 4, 5}, {9, 8, 7}};

for(auto i = 0; i < v.size(); i++)
{
    sort(v[i].begin(), v[i].end());

    for(auto j = 0; j < v[i].size(); j++){
        cout << v[i][j] << " ";
    }

    cout << endl;
}
```

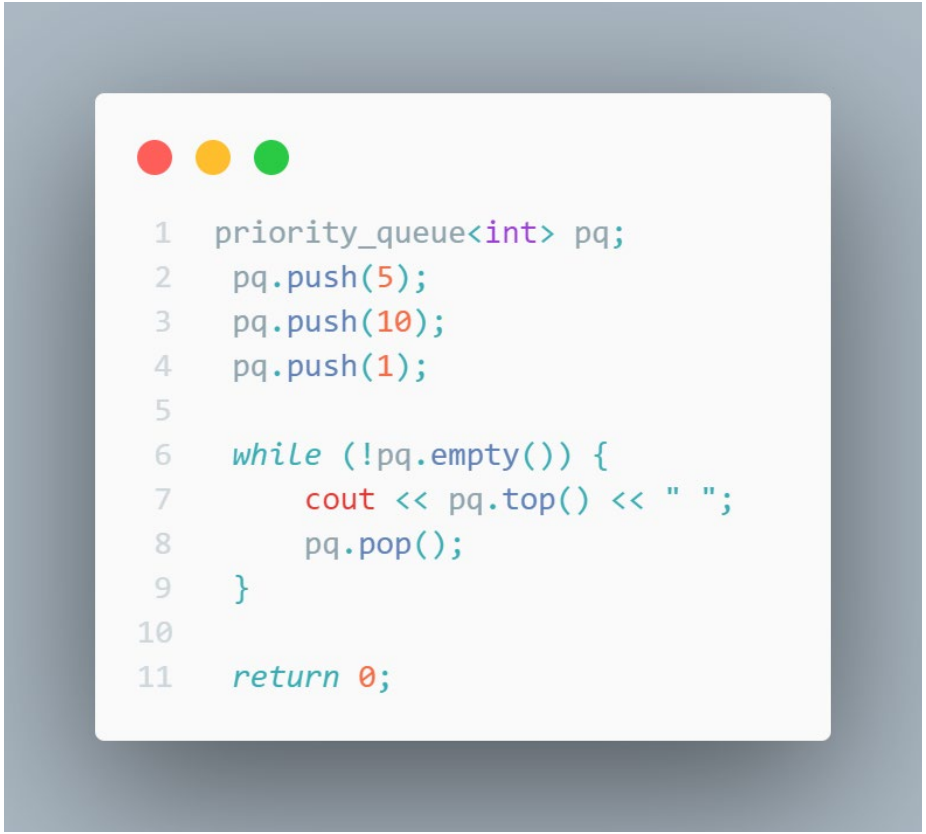
HashMap

```
map<string, int> age;
 age["Gigel"] = 25;
 age["Jănel"] = 30;
 age["Ionel"] = 35;


 for (auto &pair : age)
     cout << pair.first << " is " << pair.second << " years old." << endl;

```

Priority Queue



```cpp
priority_queue<int> pq;
 pq.push(5);
 pq.push(10);
 pq.push(1);

 while (!pq.empty()) {
     cout << pq.top() << " ";
     pq.pop();
 }

 return 0;
```

Project Structure

Project structure

Odată cu creșterea complexității și dimensiunii, un proiect devine din ce în ce mai greu de întreținut, scalat și dezvoltat. Din acest motiv, anumite “convenții” sunt stabilite pentru ca codebase-ul să poată ajunge într-un mod scalabil la dimensiuni mari.

Majoritatea limbajelor de programare și framework-urilor urmează o structură generică în ceea ce privește structurarea aplicațiilor, din punct de vedere al funcționalității și al layer-urilor.

Structura în C/C++

Pentru aplicații de mici dimensiuni (ex.: un laborator de POO), putem include tot într-un singur fișier:

- Imports
- Declarations
- Implementations

În schimb, pentru aplicații mari, este imposibil să urmăm structura aceasta.

1. Header files

În header files (.h/.hpp) ne declarăm clasele, membrii și funcțiile lor, precum și importul altor headere necesare.

Folosind headere, putem vedea ușor structura claselor, precum și amplasarea lor în memorie.

```cpp
#ifndef OOP_ANALYTICS_H // Opening include guard
#define OOP_ANALYTICS_H

#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <steps.h>
#include <utility>
#include <flowRunner.h>  // other header files

class Observer {
public:
    string flow_name;
    int times_started = 0;
    int times_finished = 0;

    void updateStarted();
    void updateFinished();

    void setFlowName(string flow_name);

    static void updateAnalytics(const Observer& observer);

    Observer();
};

#endif// Closing include guard
```

2. Implementare

În .cpp-uri implementăm logica funcțiilor declarate în header file-uri.

Pentru ca proiectele multi file să poată fi rulate folosind această structură, este nevoie de un **build system.**

```cpp
#include "analytics.h" // Include the header file

using namespace std;

///@brief Observer class
Observer::Observer(){
    this->times_finished = 0;
    this->times_started = 0;

}

void Observer::setFlowName(string new_flow_name) {
    this->flow_name = std::move(new_flow_name);
}

void Observer::updateStarted() {
    this->times_started += 1;
}

void Observer::updateFinished() {
    this->times_finished += 1;
}

void Observer::updateAnalytics(const Observer& observer) {
    string flow_list = "FlowList.csv";
    vector<vector<string>> flow_list_content = FlowRunner::readCSV(flow_list);
    vector<vector<string>> config_file_content;
    vector<vector<string>> run_file_content;
    int errors;
    string flow_name = observer.flow_name;

    ofstream flow_list_file;
    flow_list_file.open(flow_list, ios::app);
```

Build Systems

Makefile

În ecosistemul C/C++ sunt utilizate Makefile-urile drept buildsystem, unde sunt setate flagurile necesare compilării.



```
CXX = g++                                   # Compiler
CXXFLAGS = -std=c++17 -Wall -Wextra -g      # Compiler Flags
TARGET = build/my_program                   # Target executable
SRCS = main.cpp utils.cpp                   # Source files
OBJS = $(SRCS:.cpp=.o)                      # Object files (replace .cpp with .o)

all: $(TARGET)                              # Default target
$(TARGET): $(OBJS)                          # Build the target
    @mkdir -p build                      # Create build directory if it doesn't exist
    $(CXX) $(CXXFLAGS) -o $@ $^          # Link object files into the final executable

%.o: %.cpp                                  # Rule to compile source files into object files
    $(CXX) $(CXXFLAGS) -c $< -o $@       # Compile .cpp to .o

clean:                                      # Clean up build files
    rm -rf build *.o                     # Remove build directory and object files

run: all                                    # Run the program
    ./$(TARGET)                          # Execute the program

.PHONY: all clean run                       # Phony targets
```

CMake

CMake este un generator de buildsystem-uri, ce vine cu multe avantaje față de a utiliza Makefile-uri ca atare:

- Independent de platformă (se adaptează la toolchain-ul folosit: Unix/Win)
- Dependențele sunt ușor de integrat (Boost, Crypto etc.)
- Suportă cross-compling.
- Suportă configurații diferite (ex. debug /release)
- Se integrează cu multe buildsystem-uri: Ninja, GNU Make, MSBuild.

```
cmake_minimum_required(VERSION 3.10)            # Minimum required CMake version
project(MyProgram VERSION 1.0)                  # Project name and version

set(CMAKE_CXX_STANDARD 17)                      # Set C++ standard to C++17
set(CMAKE_CXX_STANDARD_REQUIRED True)           # Enforce C++ standard
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wextra -g") # Add compiler flags

set(SOURCES main.cpp utils.cpp)                 # List of source files

add_executable(my_program ${SOURCES})           # Define the executable target

target_include_directories(my_program PUBLIC .) # Add include directories
```

Project structure

La cel mai amplu nivel, aceste structuri evoluează la **architectural design patterns.**

Majoritatea framework-urilor sunt bazate pe asemenea conventii, “boilerplate driven”.

Spre exemplu:

- MVC (**M**odel **V**iew **C**ontroller): Spring Boot, Angular, Django
- MVVM (**M**odel **V**iew **V**iew**M**odel): Kotlin, Flutter
- MVP (**M**odel **V**iew **P**resenter): Android Dev